

Gry: efektywność i symulacje

Paweł Rychlikowski

Instytut Informatyki UWr

23 kwietnia 2026

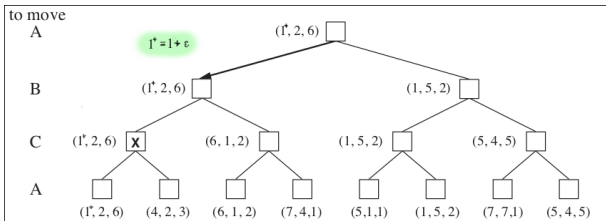


A jak grać w brydża, bądź inną grę karcianą?

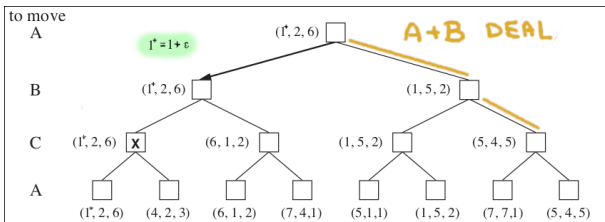
Idea (do rozwinięcia na ćwiczeniach)

losowanie układu kart i gra w otwarte karty dla wylosowanego układu, czynności powtarzamy wiele razy

Gry z większą liczbą uczestników



Gry z większą liczbą uczestników



Strategia maksymalizująca korzyść pojedynczego gracza w oczywisty sposób nieoptymalna (A mógłby się dogadać z B).

Terminologia

Wobec odpowiednika minimaksa dla gier wieloosobowych używa się nazwy $\max^n$.

Inne zagadnienia:

- Kwestie sojuszów, zrywania sojuszów, budowania wiarygodności.
- Czasem używa się: **paranoidalnego założenia** – gra wieloosobowa staje się jednoosobową, w której oni wszyscy chcą mi zaszkodzić.

Uwaga

Początki gier są podobne (bo rozpoczynamy z tego samego stanu startowego)

Z tego wynika, że:

- Możemy np. poświęcić parę godzin, na obliczenie najlepszej odpowiedzi na każdy ruch otwierający.
- Możemy „rozwinąć” początkowy kawałek drzewa (od któregoś momentu tylko dobre odpowiedzi oponenta)
- Możemy skorzystać z literatury dotyczącej początków gry (obrona sycylijska, partia katalońska, obrona bałtycka, i wiele innych)

Pytanie

Jak wykorzystać drzewo trie i duży zbiór gier (np. z chess.com)?

- Stany mogą się powtarzać (również z zeszłej partii naszego programu).
- Czasem do stanu możemy dojść na wiele sposobów (zwłaszcza, jak ruchy są od siebie niezależne)
- Jeżeli mamy oceniony stan z głębokością 6 i dochodzimy do niego z głębokością 3, to opłaca się wziąć tę bardziej precyzyjną ocenę (w dodatku bez żadnych obliczeń).

Uwaga

Potrzebny nam jest efektywny sposób pamiętania sytuacji na planszy.

Tabele transpozycji

- Zapamiętywanie pozycji powinno być efektywne pamięciowo i czasowo.
- Używa się następującego schematu kodowania (**Zobrist hashing**):
 - Mamy zdania typu: **biały goniec jest na g6 (WB-G6)**, **czarny król jest na b4 (BK-B4)**, itd (12×64)
 - Każde z nich dostaje losowy ciąg bitów (popularny wybór: **64 bity**)
 - Planszę kodujemy jako **xor** wszystkich prawdziwych zdań o tej planszy.
 - Zauważmy, jak łatwo przekształca się te kody:
nowy-kod = stary-kod **xor** wk-a4 **xor** wk-b5
to ruch białego króla z a4 na b5

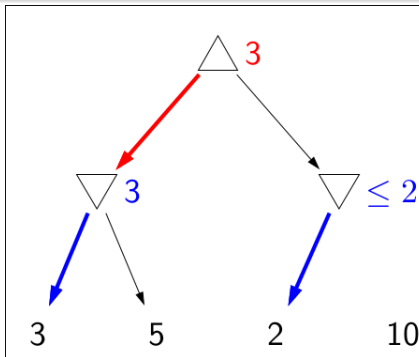
Uwaga

Często nie przejmujemy się konfliktami, uznając że nie wpływają w znaczący sposób na rozgrywkę.

Obcinanie fragmentów drzew

Idea

nie zawsze musimy przeglądać całe drzewo, żeby wybrać optymalną ścieżkę

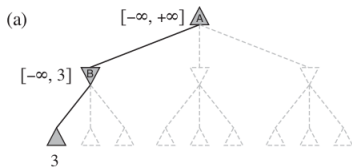


Źródło: CS221, Liang i Ermon

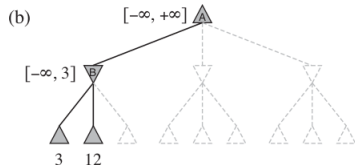
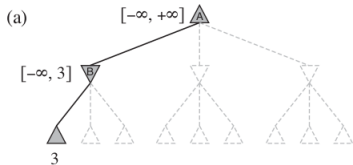
Mamy: $\max(3, \leq 2) = 3$

- Jeżeli możemy udowodnić, że w jakimś poddrzewie nie ma optymalnej wartości, to możemy pominąć to poddrzewo.
- Będziemy pamiętać:
 - α – dolne ograniczenie dla węzłów MAX ($\geq \alpha$)
 - β – górne ograniczenie dla węzłów MIN ($\leq \beta$)

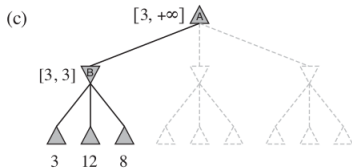
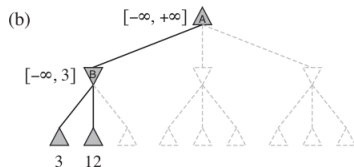
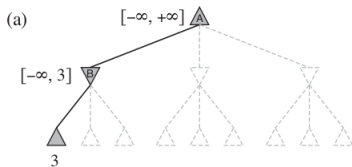
Alfa-Beta w akcji



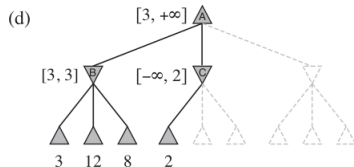
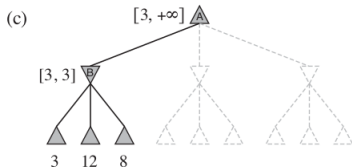
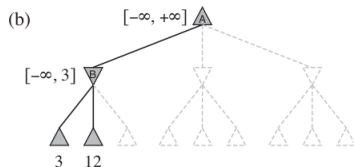
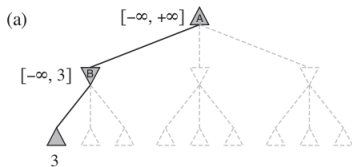
Alfa-Beta w akcji



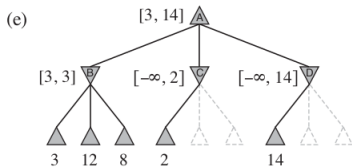
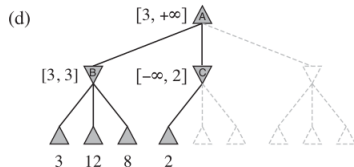
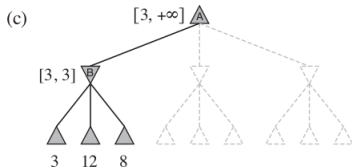
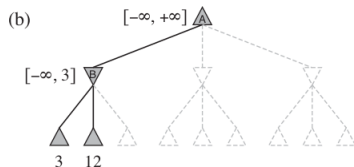
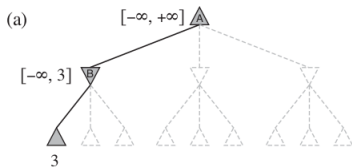
Alfa-Beta w akcji



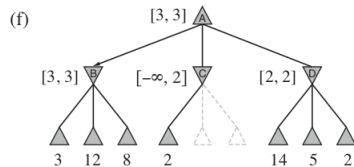
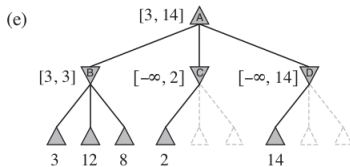
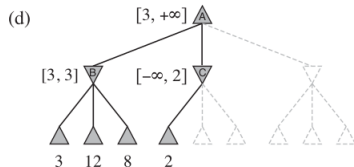
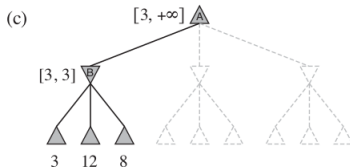
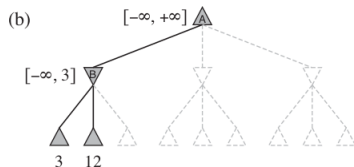
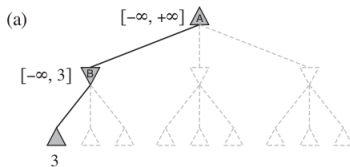
Alfa-Beta w akcji



Alfa-Beta w akcji



Alfa-Beta w akcji



- Będziemy pamiętać:
 - α – dolne ograniczenie dla węzłów MAX ($\geq \alpha$)
 - β – górne ograniczenie dla węzłów MIN ($\leq \beta$)

Algorytm A-B

```
def max_value(state, alpha, beta):  
    if terminal(state): return utility(state)  
    value = -infinity  
  
    for statel in [result(a, state) for a in actions(state)]:  
        value = max(value, min_value(statel, alpha, beta))  
        if value >= beta:  
            return value  
        alpha = max(alpha, value)  
    return value  
  
def min_value(state, alpha, beta):  
    if terminal(state): return utility(state)  
    value = infinity  
  
    for statel in [result(a, state) for a in actions(state)]:  
        value = min(value, max_value(statel, alpha, beta))  
        if value <= alpha:  
            return value  
        beta = min(beta, value)  
  
    return value
```

Kolejność węzłów

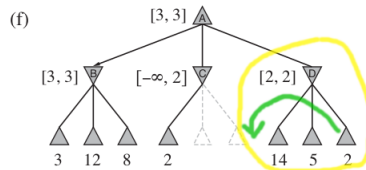
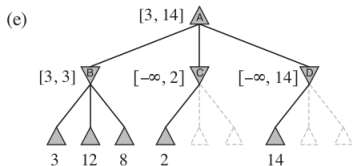
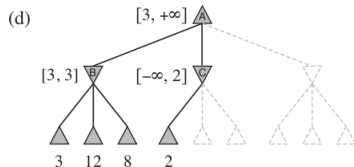
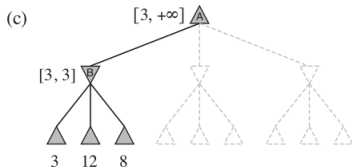
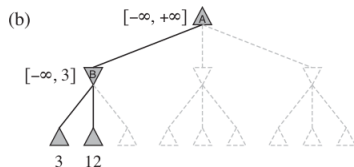
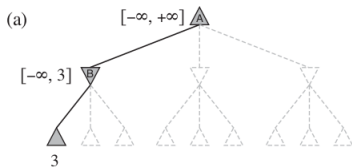
- Efektywność obcięć zależy od porządku węzłów.
- Dla losowej kolejności mamy czas działania $O(b^{2 \times 0.75d})$ (czyli efektywne zmniejszenie głębokości do $\frac{3}{4}$)

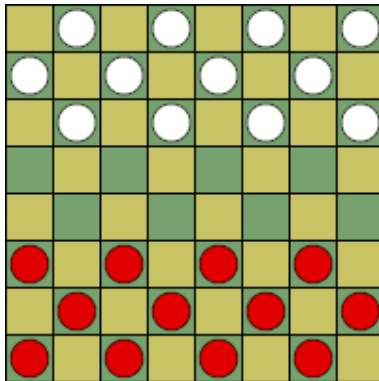
Dobrym wyborem jest użycie funkcji `heuristic_value` do porządkowania węzłów.

Uwaga

Warto porządkować węzły jedynie na wyższych piętrach drzewa gry!

Zmiana kolejności wpływa na efektywność





- Ruch po skosie, normalne pionki tylko do przodu.
- Bicie obowiązkowe, można bić więcej niż 1 pionek. Wybieramy maksymalne bicie.
- Przemiana w tzw. damkę, która rusza się jak goniec.

- Pierwszy program, który „uczył” się gry, rozgrywając partie samemu ze sobą.
- Autor: Arthur Samuel, 1965

Przyjrzyjmy się ideom wprowadzonym przez Samuela.

- ❶ Alpha-beta search (po raz pierwszy!) i spamiętywanie pozycji
- ❷ Przyspieszanie zwycięstwa i oddalanie porażki: mając do wyboru dwa ruchy o tej samej ocenie:
 - wybieramy ten z dłuższą grą (jeżeli przegrywamy)
 - a ten z krótszą (jeżeli wygrywamy)

Idea uczenia przez granie samemu ze sobą

Wariant 1

Patrzymy na pojedynczą sytuację i próbujemy z niej coś wydedukować.

Wariant 2

Patrzymy na pełną rozgrywkę i:

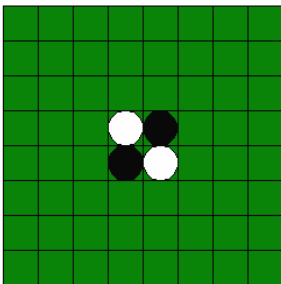
- a) Jeżeli wygraliśmy, to znaczy, że nasze ruchy były dobre a przeciwnika złe
- b) W przeciwnym przypadku – odwrotnie.

W programie Samuela użyty był wariant pierwszy. Program starał się tak modyfikować parametry funkcji uczącej, żeby możliwie przypominała **minimax** dla głębokości 3 z bardzo prostą funkcją oceniającą (liczącą bierki).

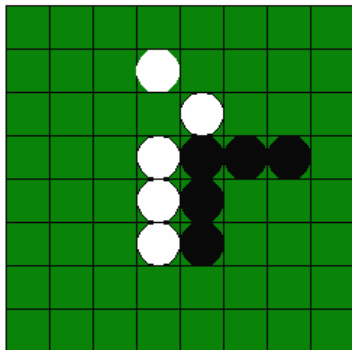
- Gra znana od końca XIX wieku.
- Od około 1970 roku pod nazwą Othello.

Nadaje się dość dobrze do prezentacji pewnych idei związanych z grami: uczenia i Monte Carlo Tree Search.

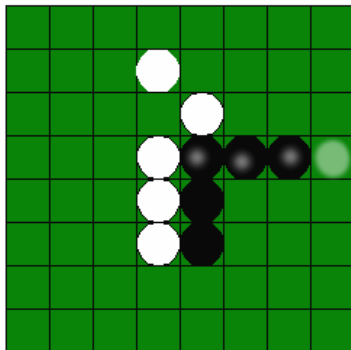
Reversi. Zasady



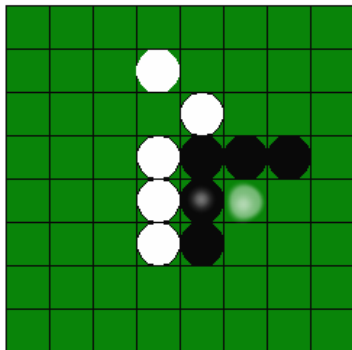
- Zaczynamy od powyższej pozycji.
- Gracze na zmianę dokładają pionki.
- Każdy ruch musi być biciem, czyli okrążeniem pionów przeciwnika w wierszu, kolumnie lub linii diagonalnej.
- Zbite pionki zmieniają kolor (możliwe jest bicie na więcej niż 1 linii).
- Wygrywa ten, kto pod koniec ma więcej pionków.



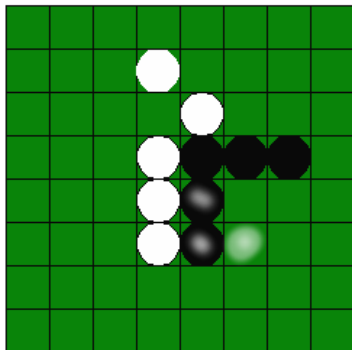
Ruch przypada na białego.



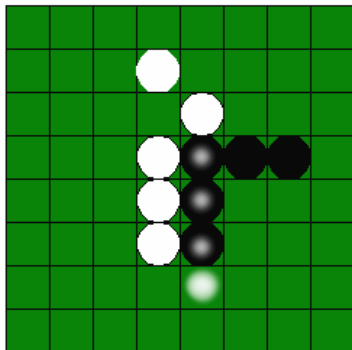
Bicie w poziomie



Bicie w poziomie



Bicie w poziomie i po skosie



Bicie w pionie

Przykładowa gra

- Popatrzmy szybko na przykładową grę.
- **Biały**: minimax, głębokość 3, funkcja oceniająca = balans pionków
- **Czarny**: losowe ruchy

Prezentacja: `reversi_show_unicode.py`

Wniosek 1

Gracz losowy działa całkiem przyzwoicie. Może to świadczyć o sensowności oceny sytuacji za pomocą symulacji.

Wniosek 2

Jest wyraźna potrzeba **nauczenia się** sensowniejszej funkcji oceniającej.

Wariant życiowy

Jesteśmy na wakacjach, jemy obiad w restauracji. Nawet smakowało. Powtarzamy, czy szukamy innego miejsca?

- Standardowy dylemat agenta działającego w nieznanym środowisku:
 1. Maksymalizować swoją korzyść biorąc pod uwagę aktualną wiedzę o świecie.
 2. Starać się dowiedzieć więcej o świecie, być może ryzykując nieoptymalne ruchy.
- Pierwsza strategia to **eksploatacja**, druga to **eksploracja**.

Jednoręki bandyta



Źródło: Wikipedia

Po pociągnięciu za rączkę, pojawia się wzorek, który (potencjalnie) oznacza naszą niezerową wypłatę.

- Mamy wiele tego typu maszyn.
- Możemy zapomnieć o wzorkach, maszyny po prostu generują wypłatę, zgodnie z nieznanym rozkładem.
- Znajomy właściciel kasyna wpuścił nas na kwadrans do sali z takimi automatami. Jak gramy?
- Bardzo wyraźnie widać dylemat eksploracja vs eksploatacja.

Wieloręki bandyta. Przykładowe strategie

- **Zachłanna**: każda rączka po razie, a następnie... ta która dała najlepszy wynik.
 - **Lepiej**: najlepszy średni wynik do tej pory
- **ϵ -zachłanna**: rzucamy monetą. Z $p = \epsilon$ wykonujemy ruch losową rączką, z $p = 1 - \epsilon$ – wykonujemy ruch rączką, która ma najlepszy **średni** wynik do tej pory.
- **Optymistyczna wartość początkowa**: inny sposób na zapewnienie eksploracji. Na początku każdy wybór obniża atrakcyjność danego bandyty.